

CI/CD Pipeline

Contents

1. [Generic questions](#)
2. [Vendasta-specific questions](#)

CI/CD automates the path from a code commit to production: build, test, scan, and deploy every change through a sequence of automated jobs. Done well, a team ships small changes many times a day with confidence. Done poorly, the pipeline is a slow, flaky bottleneck the team works around.

Generic questions

1. What is CI/CD?

CI/CD automates the path from a commit to production. **CI (Continuous Integration)** builds and tests every commit; **CD** takes a passing build and either makes it ready to release (**Continuous Delivery**) or releases it automatically (**Continuous Deployment**).

2. Why do we need CI/CD, and what problems does it solve?

It lets a team ship small changes often and safely, instead of risky big-bang releases. Every commit is built and tested, so integration bugs are caught early; slow manual steps disappear; and releases become repeatable.

Real-world example: a team without CI/CD releases by hand once a month. On release night someone runs the build on their own machine, forgets a config step, and a bug ships. With 200 commits bundled into that release, no one can tell which change broke it. With CI/CD, every push is built, tested, and deployed in minutes, so that same bug fails the pipeline on the exact commit that caused it and never reaches production.

3. Can a project work without CI/CD?

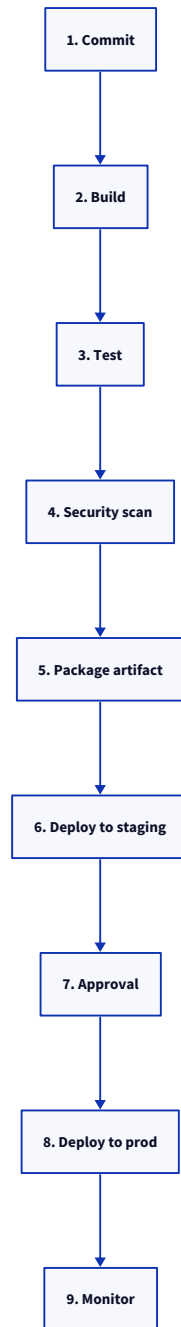
Yes, with manual builds, tests, and deploys, but it does not scale. Releases get slow, risky, and error-prone as the team and codebase grow. CI/CD pays off once more than one person ships regularly.

4. What are the disadvantages of CI/CD?

Upfront setup and ongoing maintenance, pipeline infrastructure cost, and a slow or flaky pipeline becoming a bottleneck the whole team waits on. It also demands discipline: good tests, small commits, and fast feedback.

5. Explain the CI/CD lifecycle.

A commit flows through build, test, scan, package, and deploy, with monitoring after. Fast, cheap checks run first; expensive ones later; deployment last. Failure at any stage stops the rest.



6. What is Continuous Integration, and why integrate frequently?

CI is **building and testing every commit as it lands**, so integration problems surface immediately. Integrating frequently keeps changes small, so conflicts and bugs are found on the commit that caused them, not after weeks of divergence.

7. What happens in a CI run: after a PR, after merge, and if CI fails?

On a **pull request**, CI builds the branch and runs tests, lint, and scans; the results gate the merge. After **merge**, CI runs again on the merged code and, if set up, triggers a deploy. If CI **fails**, the pipeline stops at that stage and the merge or deploy is blocked until the author fixes it. Every commit triggers CI so nothing

merges untested.

8. What is build automation and build verification?

Build automation produces the runnable artifact from source with no manual steps (compile, bundle, containerize). **Build verification** is the quick set of checks (it compiles, unit tests pass, it starts) that confirm the build is sound before anything heavier runs.

9. Continuous Delivery v/s Continuous Deployment.

Both keep every passing build releasable. The difference is the production gate.

Aspect	Continuous Delivery	Continuous Deployment
Production release	A human approves	Automatic on a green build
Best for	Regulated or high-risk releases	Fast-moving products with strong tests

They coexist: many teams auto-deploy to dev and staging (deployment) and require approval for prod (delivery). Avoid full Continuous Deployment when a bad release is very costly or needs sign-off: banking, healthcare, aviation, and other regulated domains.

10. What is a pipeline, and what are its stages?

A pipeline is the ordered set of automated jobs a change runs through; failure at any stage stops the rest.

Stage	What it does
Build	Compile and produce the artifact
Test	Unit, integration, and more
Security scan	SAST, dependency, and container scans
Package	Build the deployable artifact or image
Artifact publish	Push the artifact to a registry
Deploy	Roll out to an environment
Smoke test	Verify the deploy actually works
Approval	Human gate before prod (delivery)
Rollback	Revert to the last good version on failure
Notify	Report status to the team

11. Parallel v/s sequential v/s conditional stages.

Sequential: stages run in order, each gating the next (build, then test, then deploy). **Parallel:** independent jobs run at once to save time (unit tests, lint, and scan together). **Conditional:** a stage runs only when a condition holds (deploy to prod only on a tag or only on main).

12. What is pipeline-as-code, and why version-control it?

The pipeline is defined in a file in the repo (YAML, a Jenkinsfile) and versioned with the code it ships. So pipeline changes go through code review, roll back with a git revert, and stay in sync with the code, instead of living as hidden UI config.

13. What happens during a build, and what are build artifacts?

A build turns source into a runnable **artifact**: compile, resolve dependencies, bundle, and often containerize. The **artifact** is what you deploy (a JAR, a Docker image, a binary); it should carry everything needed to run plus a version, but not source or secrets. Source is the input; the artifact is the immutable output.

14. Immutable artifacts: why build once and never rebuild for production?

Build the artifact once and promote the same one through every environment. Rebuilding for production risks a different binary (moved dependency versions, a different build machine), so what you tested is not what you ship. Build once, tag it, and deploy that exact artifact to staging then prod.

15. How do you make builds fast and reproducible?

Fast: **cache dependencies**, use **incremental builds** (rebuild only what changed), and run independent steps in **parallel**. Reproducible: **pin dependency versions** with lockfiles, fix the build environment (a container image), and avoid anything nondeterministic, so the same source always yields the same artifact.

16. Why are automated tests important, and what test types run in CI?

They are the safety net that lets you ship often: a change is only as safe as the tests guarding it. Common types, cheap and fast first:

Test	What it checks
Unit	One function or class in isolation
Integration	Modules working together, e.g. with a database
Component	One service in isolation
API	An endpoint's contract and behaviour
UI	The interface renders and responds
Smoke	The build basically works after deploy
Regression	Old bugs stay fixed

Test	What it checks
End-to-end	A full user flow across the system
Performance	Latency and throughput under load
Security	Known vulnerabilities and unsafe patterns
Contract	Two services agree on the interface
Mutation	Whether the tests actually catch injected bugs

17. Test coverage v/s code coverage, and should deployment depend on coverage?

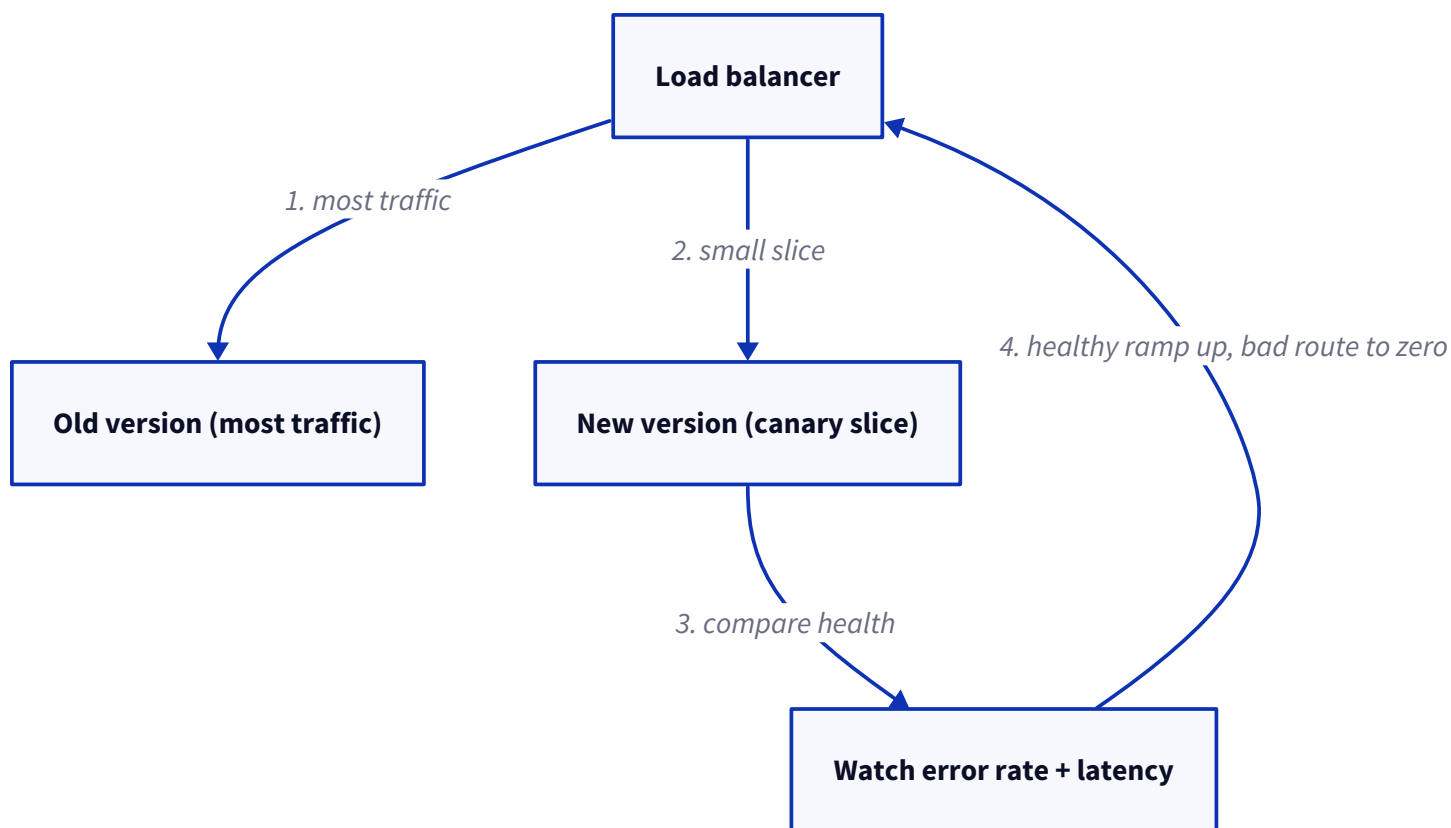
They are used interchangeably; **code coverage** is the percentage of code executed by tests. Treat it as a **signal, not a gate**: high coverage does not mean good tests, and a hard percentage gate invites gaming. Block on failing tests and critical-path coverage, not on a blanket number.

18. Code review and automated review; why rerun CI after review?

Code review is a human checking correctness, design, and risk; **automated review** is lint, static analysis, and scans doing the mechanical checks. You **rerun CI after review** because the base branch may have moved or review changes were pushed, so you test the exact code that will merge, not an older version.

19. What are the deployment strategies?

- **Recreate**: stop the old version, start the new. Simplest, but causes downtime.
- **Rolling**: replace instances a few at a time. No downtime, but a slow rollout with mixed versions running.
- **Blue-green**: run the new version (green) beside the old (blue), switch the router when green is ready; instant rollback by switching back.
- **Canary**: send a small slice of traffic to the new version, watch metrics, then ramp to 100%. Limits blast radius.
- **Shadow**: mirror real traffic to the new version without serving its responses, to test under real load.
- **A/B**: route user segments to different versions to compare behaviour or business metrics (an experiment, not just a release).
- **Feature flags**: deploy code dark and toggle the feature at runtime, decoupling deploy from release.



Strategy	Downtime	Rollback	Blast radius
Recreate	Yes	Redeploy old	All users
Rolling	No	Slow, roll back instances	Grows during rollout
Blue-green	No	Instant (switch back)	All-or-nothing at switch
Canary	No	Route slice to zero	Small slice
Feature flag	No	Toggle off	Whoever the flag is on for

20. What is rollback: automatic v/s manual, and roll-forward v/s rollback?

Rollback returns to the last known-good version. **Automatic** rollback triggers on failed health checks or bad metrics; **manual** is a human decision. **Roll-forward** ships a fix forward instead of reverting, preferred when the old version is also broken or a revert would lose data, such as after a schema migration.

21. Why use Docker in CI/CD?

It makes the build and runtime **reproducible and portable**: the same image runs on the CI runner, staging, and prod, so “works on my machine” goes away. The image is the immutable artifact you build once and promote.

22. Common CI/CD tools, and how do you choose?

Tool	Note
GitHub Actions	YAML in the repo, largest mindshare, tight GitHub integration
GitLab CI	Built into GitLab
Jenkins	Self-hosted, very flexible, common in legacy enterprises
Google Cloud Build	GCP-native, YAML
CircleCI	SaaS

Choose by where your code and infrastructure already live, hosted v/s self-hosted needs, and how much flexibility versus convenience you want.

23. Why monitor after deployment, and what do you watch?

Because a green pipeline only means the tests passed, not that production is healthy. Watch **error rate**, **latency (p95/p99)**, **CPU and memory**, **logs**, and **traces**, with **alerts** on user-visible symptoms. Tie alerts to **SLOs** (targets like 99.9% success) measured by **SLIs** (the actual metric), so you page on real problems, not noise.

24. CI suddenly starts failing for all builds. How would you investigate?

If every build fails at once, suspect something **shared**, not the code. Check a recent change to the **pipeline config**, a **base image or dependency** that moved, an expired **credential or token**, the **CI runner or service** being down, or a hit quota. Look at what changed just before the failures started, and whether a previously green commit now also fails.

25. How would you deploy with zero downtime?

Never stop all instances at once. Use **rolling**, **blue-green**, or **canary** so old and new run together during the switch, put a **load balancer with health checks** in front so traffic only reaches ready instances, and make changes **backward-compatible**, especially database migrations, so both versions work during the rollout.

26. How would you implement a canary for a high-traffic service?

Deploy the new version beside the old, route a **small slice of traffic** to it (1 to 5%), and **compare error rate and latency** against the old. If healthy, **ramp gradually** (5, 25, 50, 100%); if not, route the slice back to zero. Automate the ramp and the abort on bad metrics so a bad release stays contained to that slice.

27. Why is canary not suitable for every application?

Canary needs **enough traffic** to get a meaningful signal from a small slice, **good metrics** to judge health, and the ability to **run two versions at once**. Low-traffic services, stateful systems that cannot run mixed versions, and apps without solid metrics do not fit; there, blue-green or a simple rollout is safer.

28. Describe your CI/CD pipeline from commit to production.

A push builds a **multi-stage Docker image** and runs **tests in parallel**. On green, the image is pushed to a registry and deployed to **staging automatically**, where smoke and integration tests run. Production is a **gated deploy of the same image**, rolled out to Kubernetes with health checks, and watched on error rate and latency after. A failed rollout reverts to the previous image. *(Adapt to your real pipeline.)*

29. Monorepo v/s polyrepo CI.

Monorepo: one repo and pipeline; easy shared tooling and atomic cross-service changes, but you must build and test **only what changed** or CI gets slow. **Polyrepo**: one repo and pipeline per service; independent and fast per repo, but cross-cutting changes span many PRs and shared tooling drifts. Monorepo needs change detection; polyrepo needs coordination.

30. Tell me about a production deployment that failed. Root cause and recovery.

A deploy went out where a **database migration was not backward-compatible**: the new code expected a column the old instances did not have, so during the rolling deploy the still-running old instances started erroring. Rolling back the code was not enough because the schema had already changed, so the real fix was to **roll forward** with a corrected version. The lesson: make migrations backward-compatible and decouple schema changes from the code that depends on them (expand, then contract). *(Adapt to your own example.)*

31. How have you handled secrets in your pipelines?

Never in the repo or the pipeline file. Keep them in a **secret store** (a secrets manager, or the CI's encrypted secrets) and inject them at run time as environment variables or mounted files, scoped to the job that needs them. **Rotate** them, and never echo them into logs.

32. Design a CI/CD platform for hundreds of microservices across multiple teams.

Provide one **shared, templated pipeline** (a golden reusable workflow) that every service adopts, with per-service config for the parts that differ, so teams do not each reinvent it. Build **only what changed**, run jobs on an **autoscaling runner pool**, publish immutable artifacts to a shared registry, and standardize deploy (same rollout, health checks, rollback) across services. Keep central guardrails (required scans, prod approval) with team autonomy inside them.

33. Thousands of deployments per day: how would you scale CI/CD and control cost?

Scale runners **horizontally with autoscaling** and use **ephemeral runners** so you pay only for work done. Cut waste: **cache** aggressively, **build only what changed**, run in **parallel**, and fail fast. Control cost with **spot or preemptible** runners for non-urgent jobs, right-sized machines, and no blind retries of flaky jobs. Queue and prioritize so urgent deploys are not stuck behind batch work.

34. If you redesigned your pipeline from scratch, what would you change and why?

I would push feedback **further left**: harder caching and parallelism for speed, and stronger pre-merge checks so fewer failures reach main. I would make every deploy a **canary with automated metric-based rollback** instead of a manual watch, enforce **backward-compatible migrations** by default, and **templatz** the pipeline so every service shares one hardened definition rather than drifting copies. (*Adapt to your own pipeline.*)

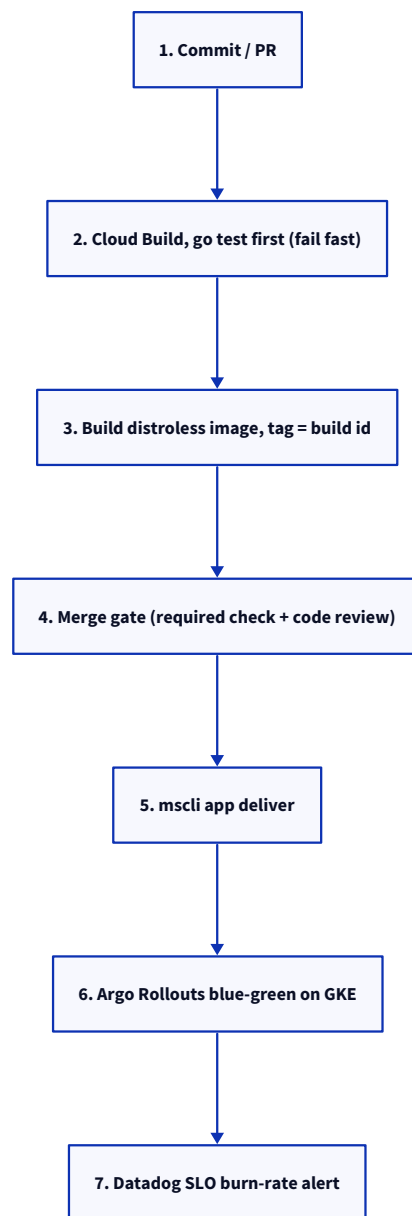
35. How would you review a PR that modifies the CI/CD pipeline?

Treat it as a **high-risk production change**. Look at **security** first: a new secret exposure, an untrusted third-party action or plugin, or a step that could leak credentials or run untrusted code (especially on a pull request from a fork). Then **reliability**: does it weaken gates (skip tests, drop the prod approval), and could it break every build at once. Then **operational**: does it change deploy or rollback behaviour, and is it reversible. Same review rigor as application code.

Vendasta-specific questions

1. Take me from a commit to production for one of your Go services. What runs on a pull request, what runs on merge, and where are the gates?

So let me walk it through the way it actually happens for one of our Go services. I push a commit, and that triggers **Cloud Build**. The first thing it does is run the Go tests, and only if those pass does it compile the binary and bake it into a small **distroless image tagged with the build id**, then push that to our registry. The same pipeline runs on a pull request and on the merge; the only difference is in the Cloud Build trigger config, not in the code. To merge, two things have to be green: that build check, and a human code review. Once it merges, the deploy is a single command, `mscli app deliver`. `mscli` reads the service's `microservice.yaml` and hands Kubernetes an **Argo Rollouts blue-green** cutover, so the new version comes up beside the old one and traffic switches over. After that, a **Datadog SLO burn-rate alert** keeps an eye on it.



What keeps PR feedback fast enough that people actually wait for it?

The trick is to only run what the change actually touched, and to run it early. On the backend the tests are scoped and the image build is cheap, because it just copies a prebuilt binary into distroless. On the frontend, which is a big Nx monorepo, we only lint, test, and build the projects the change affected, not the whole thing. And because tests run before the image build, a failure comes back quickly instead of after a slow build.

Which checks block merge, and how do you order stages to fail fast?

Two gates block the merge: the required Cloud Build check, which is tests plus a clean build, and code review. For fail-fast, we put `go test` as the very first step, before the compile and the image build, so a broken test stops everything straight away rather than wasting a build.

How do you stop the pipeline itself from becoming the bottleneck?

Mostly by not doing wasteful work: test first, a tiny distroless image with no heavy compile inside it, and on the frontend only building what changed. And the deploy is hands-off through mscli and Argo Rollouts, so nobody is sitting there babysitting a rollout, which is usually where teams lose time.

2. Tell me about a bad deploy or a pipeline failure you handled. What broke, how did you catch it, and what did you change so it could not recur?

Sure. The one that taught us something was a deploy that passed all the health checks but still degraded a metric. Here is the thing about our blue-green setup: it **auto-promotes** once the new pods are ready, and this service did not have a metric gate configured, so as far as the rollout was concerned everything was fine, the process was up and readiness was green. But the change had pushed up the error rate on one endpoint. What caught it was the **Datadog SLO burn-rate alert**, not the rollout itself. We rolled back by redeploying the previous image tag, and it recovered. *(I would swap in my own real example in the room.)*

Detection: SLO alert, canary, or user report?

It was the SLO burn-rate alert on Datadog. There was not a canary metric gate on that rollout, and readiness probes only tell you the process is alive, not that it is actually healthy, so the alert firing on the error budget is what told us something was wrong.

The rollback or forward-fix in the moment.

In the moment we just re-applied the previous image tag through mscli, which does a fresh blue-green cutover back to the last good version. Because every image is tagged by build id, the previous good build is always sitting there ready to redeploy, so it is quick.

The systemic fix

The lesson was that health checks alone are not enough. mscli can run a **post-promotion analysis** against a Datadog metric and auto-abort a bad rollout before it takes all traffic; we just had not turned it on for this service. Switching it on moves us from “watch the dashboard after every deploy” to letting the rollout gate itself.

3. During the Python-to-Go strangler migration you ran both paths in parallel behind flags. How did the pipeline support shifting traffic gradually and rolling back one capability?

Honestly, I want to be straight about this one, because “shifting traffic behind flags” sounds cleaner than what actually happened in the code. It was not a live percentage traffic split. What we did was roll the migration out **partner by partner**: a feature flag on the Partner service, scoped per partner, decided whether that partner ran the new path, and turning it on kicked off a **one-time Temporal migration** for them. Meanwhile a **Pub/Sub events bridge** kept the new Go store in sync with what the old Python monolith was writing. If we needed to back a partner out, we just turned the flag off. If there was ever a request-level traffic split between the two stacks, that lived at the ingress layer, not in the app code.

Routing a percentage of traffic between stacks.

There is not a percentage router in our code. The way we controlled the rollout was per partner through the flag, so you switch one partner on, watch it, then switch on more. A true request-level split would live in the ingress or load balancer, which is infra, not these repos.

Keeping the two stacks independently deployable.

That part fell out naturally, because each stack is its own service with its own pipeline. The Python monolith and the Go services deploy separately, each through its own Cloud Build and blue-green rollout, so releasing one never blocks the other. That is what let us move one capability at a time.

Verifying parity before cutover, with the events bridge staying consistent during a partial rollout.

For parity, the old side publishes every draft change on Pub/Sub and the new side subscribes, so both stores are tracking the same writes and you can compare them. The consumers are **idempotent**, for instance a delete of a draft that is already gone is treated as success, and the Temporal migration keeps heartbeat progress, so if a step retries it does not reprocess drafts it already did. That is what keeps the bridge consistent while some partners are still on the old path and some are on the new.

4. The frontend lives in the galaxy monorepo. How do you keep CI fast when one repo holds many apps, so a small change does not rebuild and retest everything?

We lean on **Nx affected** for this. CI works out a base and head, basically the branch compared against master, and Nx uses its project graph to figure out which projects the change actually touched plus anything that depends on them, and then only lints, tests, and builds those. Even the publish step is scoped that way, so only the apps that changed get deployed.

Computing affected targets, and remote build caching.

The affected set comes from the git diff plus the Nx project graph. On caching I should be honest: we use Nx's **local cache** but we have Nx Cloud switched off, so there is no shared remote cache across CI runs. Every Cloud Build run starts on a fresh worker, so it does not reuse the last run's results. If build time ever became the real pain point, a remote cache would be the obvious next lever.

Test isolation between apps in the monorepo.

Each app is its own Nx project with its own Jest config, all extending one shared preset. When we run affected tests in parallel, each app's suite runs as its own process, so one app's global state or a failure in it does not bleed into another.

What goes wrong when the dependency graph is inaccurate?

This is the catch with affected. If the graph misses a dependency, you under-test: a change ships without rebuilding something that actually depended on it, and you get a false green. If the graph over-declares dependencies, you over-build and CI slows down touching apps that did not really change. So the whole

thing is only as good as the accuracy of that graph.

5. Proto changes flow through vendastaapis, Cloud Build, and generated-protos-go before a service consumes them. How does that pipeline work, and how do you stop a breaking change from shipping?

So a proto change goes into **vendastaapis**, and pushing it triggers Cloud Build. That runs **buf lint** for style and naming plus our own scope linter, generates the **Go stubs**, and pushes them into the generated-protos-go repo. On master it also tags each proto package with its own semver. A Go service only picks up the change when it bumps that package's version in its `go.mod` and runs `go get`, so nothing lands on a consumer automatically.

Honest gap

We do not actually detect breaking changes in this pipeline. `buf.yaml` declares breaking rules, but `buf breaking --against` is never run, only `buf lint`. So a removed or renumbered field can pass CI and get published. The only real safety net is that consumers pin an exact version and upgrade on purpose. The fix I would add is a `buf breaking` step, and a PR gate, in vendastaapis.

Protobuf backward and forward compatibility rules.

The rules I keep in mind: never reuse or renumber a field number, and never change a field's type. You add, you do not remove, and if you must retire a field you mark it `reserved` or deprecated. New fields are optional, and a receiver just ignores fields it does not know about, which is what gives you forward compatibility. Stick to that and old and new code talk to each other fine.

Detecting a breaking change in CI, before consumers pull it.

Right now, honestly, we do not catch it automatically, which is the gap I mentioned. The right control is a `buf breaking` check against the last published version in the vendastaapis pipeline, ideally as a PR gate, so a wire-incompatible change is blocked before it is ever tagged and published.

Coordinating a producer and consumer rollout when they deploy separately.

Because the consumer upgrades on its own schedule, you do it **expand-then-contract**. The producer ships the additive change first, something backward-compatible like a new field or RPC, the consumers move over at their own pace, and only once everyone is off the old field do you actually remove it. You never pull something out from under a consumer that is still reading it.

6. A deploy goes bad in production. Walk me through your rollback: how fast, what mechanism, and how do you avoid making it worse?

How fast the rollback is really depends on timing. With blue-green, the old version's pods hang around for a **scale-down grace window**, so if we catch it inside that window it is near-instant, we just abort the rollout or switch traffic back to the old pods. Once those old pods are gone, we roll back by re-applying the previous image tag through msccli, which runs a fresh blue-green cutover to the known-good image. Since every image is tagged by build id, the last good one is always a specific artifact we can redeploy.

Rolling back the image v/s flipping a feature flag.

Rolling back the image reverts everything in that deploy. Flipping a feature flag reverts just the one feature and does not need a redeploy, so it is faster and more surgical, but only if the risky change was behind a flag in the first place. Our flags, the Partner flag service scoped per partner, are more for gating features and migrations than for the deploy itself, so image rollback is the general tool and a flag is the precise one when it happens to apply.

A rollback made unsafe by a data or schema migration: what then?

This is the case you have to watch for. If the new version changed the schema or started writing data in a new shape that the old version cannot read, then rolling the code back actually breaks it against the changed data. In that spot you **forward-fix** instead, ship a corrected version quickly rather than revert. The way to avoid the whole problem is backward-compatible migrations, expand-then-contract, so both versions work against the same schema and a rollback stays safe.

Forward-fix v/s rollback: how do you decide under pressure?

My default is rollback, because if the previous version is known-good it is the fastest way back to healthy. I switch to forward-fix when a rollback would break against data that has already been migrated, or when the old version is also broken. So the first question I ask under pressure is “did this deploy touch data or schema”, because that is what decides whether rolling back is even safe.